



Universidad Autónoma de Yucatán

Facultad de Ingeniería

**Intelligent Resource Management and Integration
of the Intel Agilex FPGA in a Reconfigurable
Cluster for High-Performance Computing**

PROTOCOL

Presentado por:

Pedro Antonio Casanova Martin

En opción al grado de:

Ingeniería Mecatrónica

Asesor:

Dr. Alejandro A. Castillo Atoche

Mérida, Yucatán, México

2026

Contents

1	Introducción	3
2	Objectives	5
2.1	General Objective	5
2.2	Specific Objectives	5
3	Background	6
4	Methodology	12
5	Activity Schedule	14
	Bibliograhpy	14

Introducción

FPGAs are integrated circuits that can be reconfigured after manufacturing, allowing developers to create customized hardware pipelines optimized for specific tasks. Compared to CPUs and GPUs, they stand out for their parallelism, low power consumption, and ability to implement specialized logic, making them valuable in data processing, artificial intelligence inference, and real-time systems [1],[2]. In high-performance computing environments, these devices are frequently used to accelerate operations that benefit from streaming data flows or fixed-function hardware.

An "FPGA farm or cluster" refers to a set of interconnected FPGA cards within a shared computing environment, similar to GPU clusters but focused on reconfigurable hardware. These infrastructures can run multiple applications simultaneously or allow several FPGAs to collaborate on a single workload. Recent research highlights how multi-FPGA coordination can accelerate tasks such as graph analytics [3], and how distributed or multi-tenant systems manage FPGA resources more efficiently [4]. Additionally, more recent cloud-oriented frameworks demonstrate how FPGA virtualization allows different users to share hardware efficiently for general-purpose workloads, including deep neural networks [5].

High-performance computing refers to the use of large-scale systems, typically clusters composed of hundreds or thousands of interconnected nodes, to solve complex problems or process massive datasets at a speed significantly higher than that of a conventional computer. Modern HPC configurations typically integrate CPUs, GPUs, and specialized accelerators, which has driven the development of architectures capable of adapting to different workloads rather than relying on a single processor type [4].

Intel Agilex is one of Intel's most modern FPGA families, designed to deliver improvements in performance, memory bandwidth, and connectivity. These devices incorporate embedded compute blocks, high-speed transceivers, and support for advanced interconnects, making them ideal for data-centric platforms and multi-accelerator systems. Their flexibility allows them to integrate efficiently into multi-FPGA environments or heterogeneous configurations [6].

A large language model (LLM) is an artificial intelligence system trained on large volumes of text, capable of understanding and generating natural language, as well as reasoning about complex problems. From these models, intelligent agents can be built—systems capable of perceiving their environment, making decisions, and executing actions autonomously to achieve a given objective. Unlike traditional automation systems, an LLM-based agent can interpret natural language instructions, plan sequences of actions, and dynamically adapt to unforeseen situations [7].

The demand for high-performance computing (HPC) in areas such as Big Data and artificial intelligence exceeds the capabilities of traditional systems, making FPGA-based acceleration a key solution [1],[2]. The critical challenge is not the availability of state of the art devices such as the Intel Agilex board, but rather their proper and scalable integration within a centralized infrastructure a gap that prevents organizations from fully exploiting their acceleration potential [8]. This research aligns with the transition toward heterogeneous architectures where CPUs, GPUs, and FPGAs operate jointly [9], and with the generational advance that the Agilex represents in performance and flexibility [6]. Additionally, the incorporation of an LLM-based agent automates dynamic workload assignment, eliminating manual intervention and optimizing resource utilization in the

farm [7]. From a technological and academic perspective, this project delivers a validated deployment model that documents challenges such as communication, resource allocation, and scalability, serving as a practical foundation for future research in distributed reconfigurable architectures [3],[10].

This thesis proposes the integration of an Intel Agilex FPGA board into a pre-existing reconfigurable computing infrastructure, referred to as an FPGA farm, which is composed of interconnected Intel Cyclone V boards. This integration encompasses a thorough analysis of the functional and interface requirements of the Agilex device, in order to establish the procedures and technical specifications necessary for its incorporation into the existing architecture. Subsequently, hardware description language (HDL) logic and the communication protocols required to ensure correct interoperability between the reconfigurable nodes and the central farm infrastructure will be developed.

As an additional component of the proposed architecture, an intelligent agent based on a large language model (LLM) will be implemented. Its function will be to analyze incoming dynamic workloads and identify their computational characteristics, with the aim of autonomously determining the most suitable FPGA node for their execution within the farm. In this way, digitally complex architectures, such as state machines or intensive control logic, will be routed to the Intel Agilex board, leveraging its advanced processing capabilities. Less computationally demanding architectures, in turn, will be assigned to the Intel Cyclone V boards, thus optimizing the overall utilization of resources available in the infrastructure.

The implementation will be carried out entirely within the available academic environment, focusing on demonstrating correct detection, configuration, and data exchange between the Agilex board and the other farm components through controlled test workloads. The LLM-based agent will operate exclusively as a classification and task-assignment module, limited to analyzing the computational characteristics of the digital architecture and determining the most appropriate node for its execution, without implementing state memorization mechanisms, temporal scheduling, history management, or any other extended functionality typical of more complex agentic systems. At this stage, the work does not contemplate multi-user support, virtualization layers, or cloud-oriented services, and all development and validation activities are subject to the available hardware, development tools, and documentation, as well as the established academic period.

Objectives

General Objective

Design and implement the intelligent resource management and integration of the Intel Agilex FPGA in a reconfigurable cluster for high-performance computing, guaranteeing low-latency communication, dynamic resource allocation, and operational stability under variable workload conditions.

Specific Objectives

- Analyze the functional and interface requirements of the Agilex FPGA board to document the procedures and specifications necessary for its integration into the existing FPGA farm.
- Develop the hardware description language (HDL) logic and communication protocols required for the integration of the Intel Cyclone V and Intel Agilex boards with the FPGA farm infrastructure
- Implement an LLM-based agent for the optimization and dynamic management of resources within the FPGA farm architecture.
- Validate the integration and communication of the Agilex board through the execution of test workloads, confirming the correct data flow with the FPGA farm components.

Background

FPGAs have consolidated themselves as a fundamental alternative for accelerating high-performance applications, given their ability to process data in parallel and adapt to various types of workloads. In the field of Big Data, tools such as Fletcher and Apache Arrow have demonstrated that these devices can increase efficiency in managing large-scale datasets [1]. They have also shown solid performance in artificial intelligence tasks, particularly in neural network inference, due to their low power consumption and flexibility [2].

Apache Arrow is specifically designed to work with large-scale tabular data structures stored in memory with a columnar orientation. By iterating over column entries in tables, the columnar format generates more efficient use of CPU caches and vector instructions compared to a row-oriented format [1].

The Figure 1 illustrates an example of Fletcher, presenting an open-source FPGA accelerator framework specifically designed on top of Apache Arrow, with the purpose not only of addressing challenges related to the integration of Big Data analytics structures, but also of solving challenges in the hardware development domain.

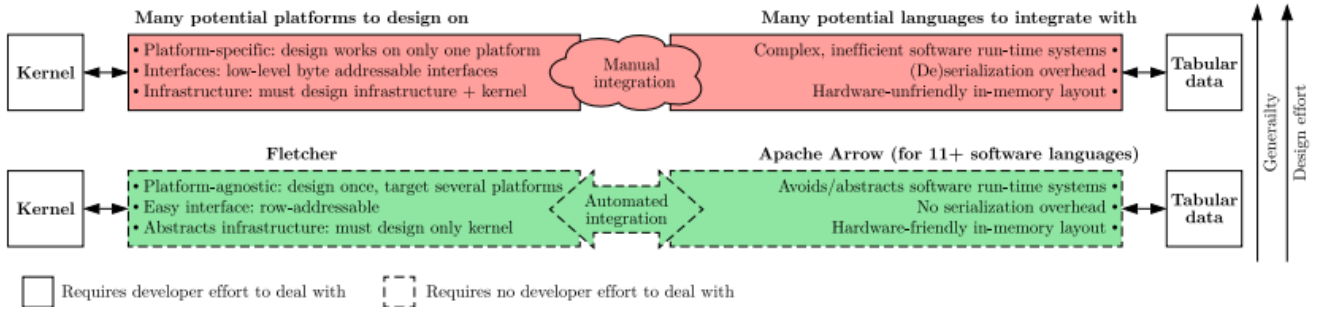


Figure 1. Advantages of the Fletcher FPGA Accelerator Framework for Big Data Systems Based on Apache Arrow [1].

FPGAs exhibit extraordinary potential for neural network acceleration due to their programmability, which allows developers to jointly evaluate both logic and algorithm, and implement specific logic for the target computation. In conventional FPGA-based accelerators, two main architectures coexist: SoC-type FPGA (System-on-Chip) and standard FPGA. The Figure 2 illustrates their structures. Every FPGA accelerator consists of two fundamental elements: the FPGA fabric and the CPU; the difference lies in whether both components are placed within the same integrated circuit. In edge computing scenarios, the CPUs used are typically ARM-architecture processors.

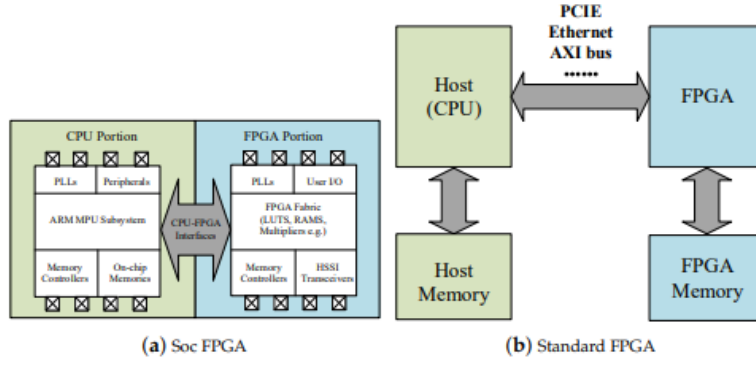


Figure 2. Two FPGA architectures. (a) represents a SoC-type FPGA (System-on-Chip), which integrates both the FPGA fabric and the CPU within the same package. (b) corresponds to a standard FPGA [2].

Additionally, various works have explored the integration of FPGAs in shared or distributed environments. Some approaches propose full-stack solutions for connecting them with cloud services and software [8], while others analyze heterogeneous cluster architectures to scale performance through the use of multiple devices [9].

The Figure 3 illustrates the RAPTOR-Xpress base board, which integrates 4 daughter cards directly interconnected with their adjacent nodes via a ring topology. Each card features a Xilinx Virtex-5 FPGA and up to 4 GB of DDR3 memory, in addition to a dedicated FPGA that acts as a PCIe interface. Multiple base boards can be interconnected through 4 high-speed connectors, each consisting of 21 full-duplex serial lines, enabling hardware resources to scale beyond the 4 daughter cards available on the board. The RAPTOR project also encompasses a custom software development environment that includes the RAPTORLIB, RAPTORAPI, and RAPTORGUI tools. These utilities assist developers by providing hardware-backed protocols, remote access, and a graphical user interface to facilitate testing. The design flow includes assistance tools for design partitioning, which consists of a manual process assisted by a graphical integrated development environment (IDE) and standard synthesis tools developed in vMAGIC [9].

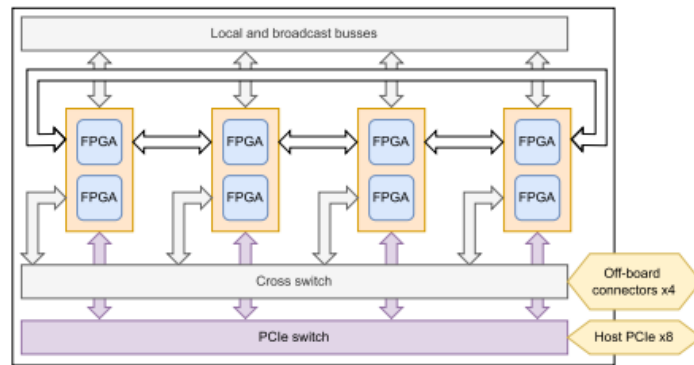


Figure 3. Simplified diagram of the RAPTOR-Xpress base board or compute unit (CU) with the daughter cards represented in orange [9].

Similarly, Cube was developed with the purpose of exploring the scalability of a cost-effective massive FPGA experimentation cluster for real-world applications. This system consisted of 8 boards hosting an 8×8 array of Xilinx FPGAs, thus forming a cluster of 512 FPGAs, as illustrated in Figure 4. The design featured a unique multi data programming

paradigm configuration that allowed all FPGAs to be configured with the same bitstream within seconds. The FPGAs were interconnected in a systolic array reaching an inter-FPGA bandwidth of up to 3.2 Tb/s, which offered significant advantages by simplifying the programming model and considerably relaxing PCB design requirements [9].

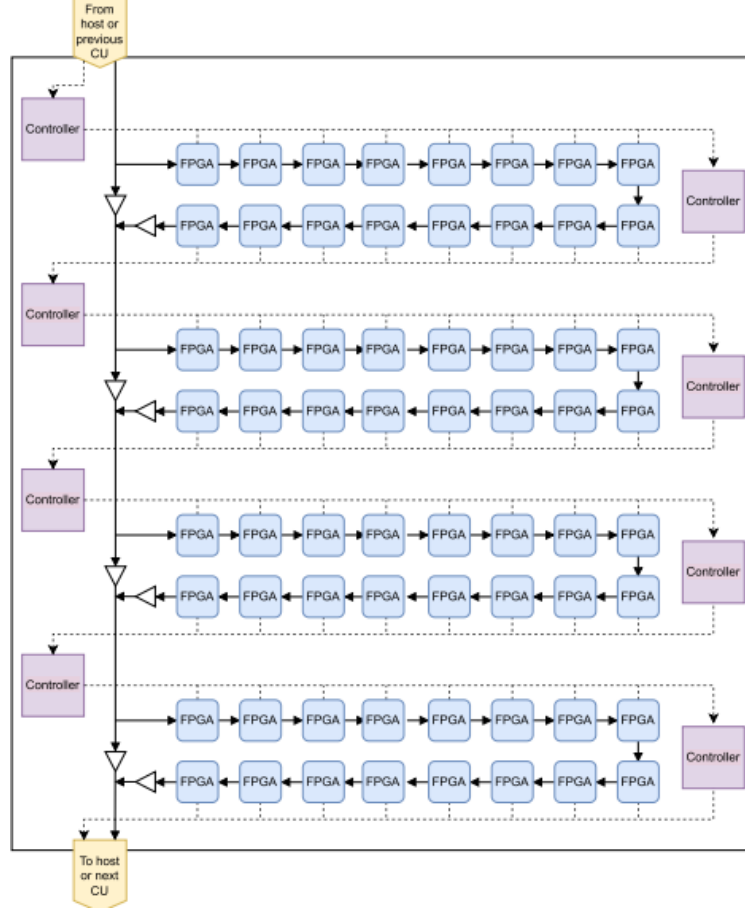


Figure 4. Cube compute unit (CU) showing the configuration controllers in purple. Dashed lines represent the control and configuration bus, while solid lines indicate the data path [9].

Intel Agilex FPGAs and SoCs enable the development of next-generation high-performance applications through superior logic fabric performance, lower power consumption, improvements in digital signal processing (DSP) functionality, and greater designer productivity compared to previous FPGA generations. The Intel Agilex FPGA addresses the enormous number of challenges posed by data-centric computing, while simultaneously opening new possibilities for enterprise and industrial environments. This technology combines a general-purpose fabric that provides flexibility with highly efficient silicon-level processing, designed to execute the specific and customized functions demanded by each market segment. The Figure 5 presents a representation of a Hyper-Register in an Intel Stratix 10 FPGA. A signal path passing through the register and another path that bypasses it can be observed. Both signal paths converge at a multiplexer (MUX), which is controlled by the configuration RAM. One of the ways in which Intel has improved the Intel Hyperflex FPGA architecture in its second generation is by accelerating the bypass path speed of the Hyper-Register. This improvement increases performance both for designs optimized for the Intel Hyperflex FPGA architecture and for those that have not been optimized for this architecture [6].

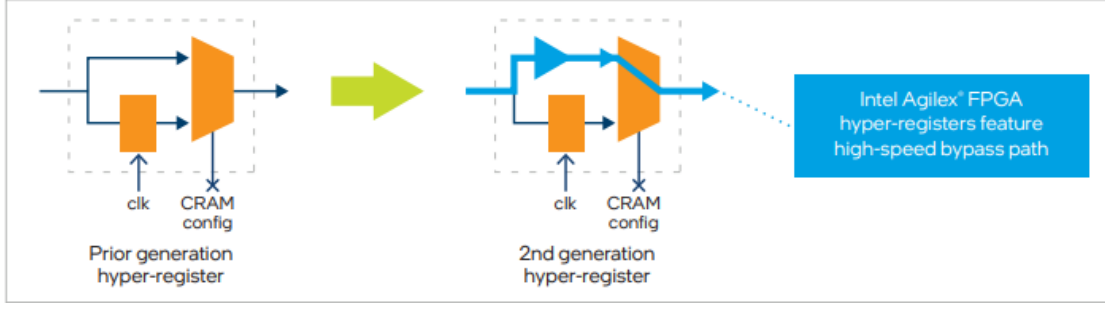


Figure 5. A high-speed bypass accelerates the Hyper-Registers to optimize logic fabric performance[6].

The Figure 6 illustrates two design examples. The upper diagram is optimized for the Intel Hyperflex FPGA architecture, while the lower one is not. The Adaptive Logic Modules (ALMs) of the Intel Agilex FPGA are represented by the large light blue blocks; the Hyper-Registers are shown in two color variants: orange for those not in use, and blue with a gray outline for those that are active. As can be observed, the upper design uses Hyper-Registers, indicating it has been optimized for the Intel Hyperflex FPGA architecture, while the lower design shows no active Hyper-Registers, evidencing the lack of optimization for this architecture [6].

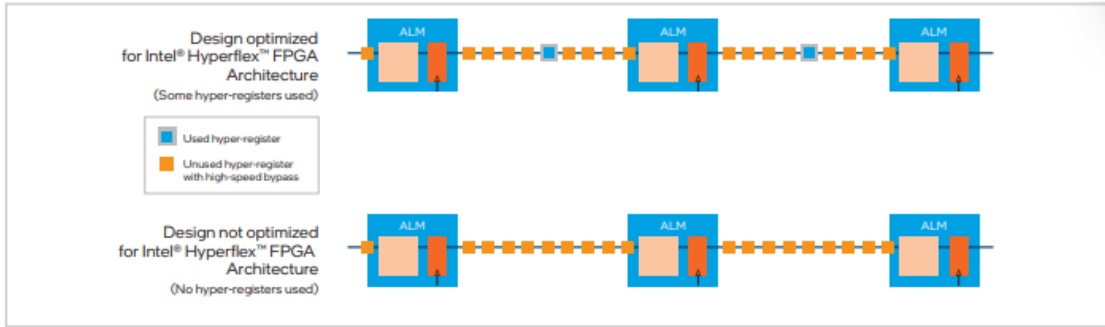


Figure 6. The upper design is optimized for the Intel® Hyperflex™ FPGA architecture, while the lower design is not [6].

Historically, FPGAs have integrated their functions monolithically, incorporating individual elements such as transceivers (XCVRs), memory, and I/O ports within the same fabric that houses the programmable logic. However, this approach has three well-defined limitations, schematized in Figure 7. First, it requires all individual FPGA functions to be developed under the same technology process node. Second, it requires a new photolithography process each time any of these individual functions is updated or improved. Third, it demands a new tape-out to obtain different combinations of resources and ratios of specific functions. Since tape out processes are time consuming and extremely costly, it is imperative to minimize them in order to accelerate the pace of innovation in product development [6].

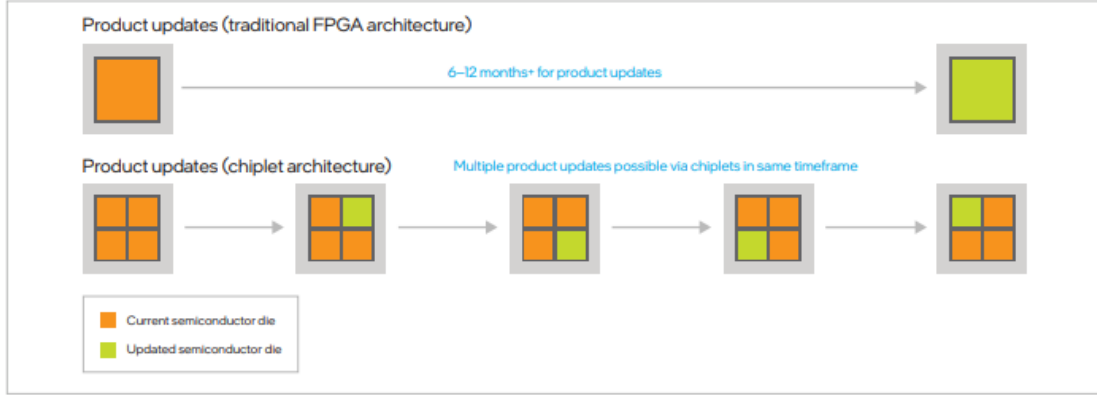


Figure 7. Traditional FPGA update cycle versus chiplet-based updates [6].

The Figure 8 describes Intel Agilex FPGA applications aimed at meeting the computing and acceleration needs of a wide range of applications, spanning from the edge to the core and the cloud. These solutions are customized for specific market segments and applications, intervening at any point where data is generated, transported, stored, or processed [6].

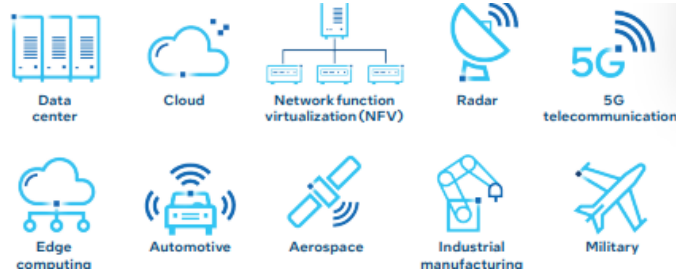


Figure 8. Application examples of the Intel Agilex FPGA [6].

In recent years, large language models (LLMs) have achieved remarkable successes, demonstrating significant potential for achieving human-like intelligence. This capability derives from leveraging massive training datasets alongside a substantial number of model parameters. From this capacity, a growing research area has emerged that employs LLMs as central controllers to build autonomous agents capable of acquiring human-like decision-making skills. Compared to reinforcement learning, LLM-based agents possess more comprehensive internal world knowledge, which facilitates the execution of better-grounded actions by the agent, even without prior training on domain-specific data. Additionally, LLM-based agents can provide natural language interfaces for interacting with humans, which is more flexible and explainable. In this regard, researchers have developed numerous promising models (see Figure 9), where the key idea consists of endowing LLMs with crucial human capabilities such as memory and planning, to make them behave like humans and complete various tasks effectively. Previously, these models were proposed independently, with limited efforts to synthesize and compare them holistically. However, a systematic compendium of this rapidly developing field is considered to be of great importance for understanding it comprehensively and helping inspire future research [7].

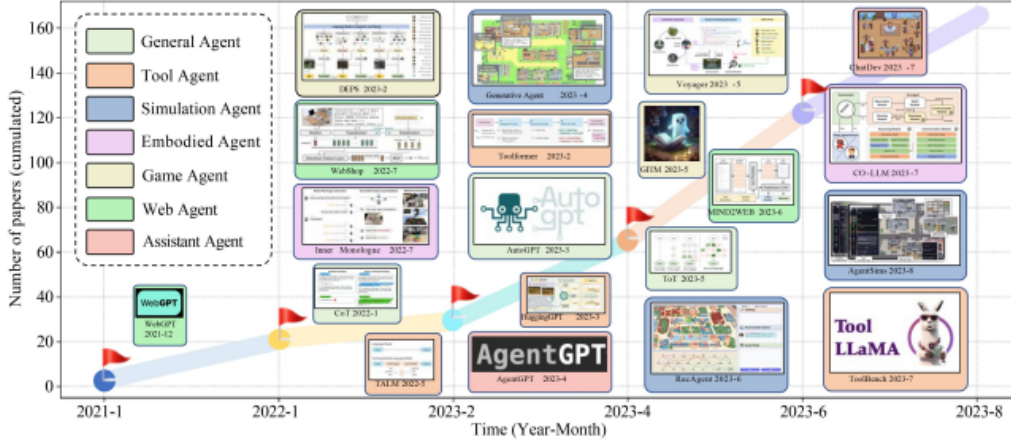


Figure 9. Illustration of the growth trend in the field of LLM-based autonomous agents. The cumulative number of papers published from January 2021 to August 2023 is presented [7].

Recent advances in Large Language Models (LLMs) have demonstrated their enormous potential to perform a wide range of tasks under the question-and-answer (QA) paradigm. However, the development of autonomous agents transcends the QA paradigm, as these require fulfilling specific roles as well as perceiving the environment and learning from it autonomously in order to evolve in a manner similar to humans. To bridge the gap between traditional LLMs and autonomous agents, a crucial aspect consists of designing rational agent architectures that assist LLMs in maximizing their capabilities. In this regard, prior works have developed various modules aimed at enhancing LLMs. In this section, a unified framework is proposed to synthesize these modules. Specifically, the general structure of this framework is illustrated in Figure 10, which is composed of a profiling module, a memory module, a planning module, and an action module. The goal of the profiling module is to identify the agent’s role. The memory and planning modules situate the agent within a dynamic environment, allowing it to remember past behaviors and project future actions. The action module is responsible for translating the agent’s decisions into specific outputs. Within these components, the profiling module impacts the memory and planning modules, and together, these three modules influence the action module [7].



Figure 10. A unified framework for the design of LLM-based autonomous agent architectures [7].

Methodology

This section presents the main process for achieving the research goals. These activities are grounded in the specific objectives of the project.

*Activity 1. **Continuous and updated literature review.*** Conduct a bibliographic review of the project’s central topics: FPGA farm architectures, Intel Agilex and Intel Cyclone V devices, communication protocols for multi-FPGA systems, hardware design using HDL, and agents based on large language models (LLMs).

*Activity 2. **Functional and interface requirements analysis.*** Analyze the technical specifications of the Intel Agilex board, documenting its power supply requirements, connectivity, communication interfaces, and compatibility with the existing FPGA farm infrastructure.

*Activity 3. **Integration architecture design.*** Define the communication architecture between the Intel Agilex board, the Intel Cyclone V boards, and the central farm infrastructure, establishing the interconnection protocols and standards to be implemented.

*Activity 4. **Hardware description language (HDL) development.*** Develop the HDL modules necessary for the configuration, initialization, and operation of the Intel Agilex board within the farm environment, ensuring its correct interoperability with the other nodes.

*Activity 5. **Communication protocol implementation.*** Implement the communication protocols required for data exchange between the Agilex board, the Cyclone V boards, and the central infrastructure of the FPGA farm.

*Activity 6. **LLM-based agent development and implementation.*** Design and implement the intelligent agent based on a large language model, configuring it to analyze the computational characteristics of incoming workloads and autonomously assign them to the most appropriate node within the farm.

*Activity 7. **Agent integration with farm infrastructure.*** Integrate the LLM agent with the farm nodes, verifying correct communication between the assignment module and the available FPGA devices.

*Activity 8. **Test workload design.*** Design and prepare representative test workloads, including state machines and combinational logic modules, to verify the functioning of the assignment system and correct execution on each node.

*Activity 9. **Integration and communication validation.*** Execute test workloads on the integrated infrastructure, verifying the correct data flow between the Agilex board and the other farm components, as well as the operational stability of the system.

*Activity 10. **Full system evaluation.*** Evaluate the integrated system in terms of operational stability, correct task assignment by the agent, and overall functionality

of the architecture under real operating conditions.

Activity 11. Results analysis and validation. Evaluate the fulfillment of the stated objectives and document conclusions, advantages, limitations, and opportunities for improvement of the implemented system.

Activity 12. Thesis writing. Write a coherent document by integrating the previously researched information, unifying it with the experimental results from hardware tests on the FPGAs and the evaluation of the LLM agent.

Activity Schedule

The following schedule determines the completion periods for the activities planned to conclude the project.

Table 1. Activity Schedule 2026

Actividades		2026											
Nº	Actividades	Ene	Feb	Mar	Abr	May	Jun	Jul	Ago	Sep	Oct	Nov	Dic
1	Revisión continua y actualizada del estado del arte												
2	Análisis de requisitos funcionales y de interfaz												
3	Diseño de la arquitectura de integración												
4	Desarrollo de la lógica de descripción de hardware (HDL)												
5	Implementación de los protocolos de comunicación												
6	Desarrollo e implementación del agente basado en LLM												
7	Integración del agente con la infraestructura de la granja												
8	Diseño de cargas de trabajo de prueba												
9	Validación de la integración y comunicación												
10	Evaluación del sistema completo												
11	Análisis y validación de resultados												
12	Redacción de la tesis												

Bibliography

- [1] J. Peltenburg, J. van Straten, M. Brobbel, Z. Al-Ars, and H. P. Hofstee, “Generating high-performance FPGA accelerator designs for big data analytics with Fletcher and Apache Arrow,” *Journal of Signal Processing Systems*, vol. 93, no. 5, pp. 565–586, 2021.
- [2] R. Wu, X. Guo, J. Du, and J. Li, “Accelerating neural network inference on fpga-based platforms—a survey,” *Electronics*, vol. 10, no. 9, p. 1025, April 2021. [Online]. Available: <https://www.mdpi.com/2079-9292/10/9/1025>
- [3] O. Jaiyeoba *et al.*, “Swift: A Multi-FPGA Framework for Scaling Up Accelerated Graph Analytics,” arXiv preprint, November 2024.
- [4] M. Yamakura, R. Takano, A. Ben Ahmed, M. Sugaya, and H. Amano, “A multi-tenant resource management system for multi-FPGA systems,” *IEICE Transactions on Information and Systems*, vol. E104D, no. 12, pp. 2078–2088, 2021.
- [5] S. Zeng, G. Dai, H. Sun, J. Liu, S. Li, G. Ge, K. Zhong, K. Guo, Y. Wang, and H. Yang, “A unified FPGA virtualization framework for general-purpose deep neural networks in the cloud,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 3, pp. 1–31, 2022.
- [6] Intel Corporation, “Intel® Agilex™ FPGAs Deliver a Game-Changing Combination of Flexibility and Agility for the Data-Centric World,” Intel Corporation, White Paper, July 2023.
- [7] L. Wang *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, 2024.
- [8] N. Tarafdar, N. Eskandari, D. Ly-Ma, and P. Chow, “Galapagos: A Full Stack Approach to FPGA Integration in the Cloud,” *IEEE Micro*, 2018.
- [9] W. F. Samayoa *et al.*, “A survey on fpga-based heterogeneous clusters architectures,” *IEEE Access*, 2023.
- [10] L. Li, H. Pan, X. Wan, K. Lv, Z. Wang, Q. Zhao, F. Ning, Q. Ning, S. Zhang, Z. Li, L. Luo, and G. Xie, “Harmonia: A Unified Framework for Heterogeneous FPGA Acceleration in the Cloud,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS ’25)*. Rotterdam, Netherlands: ACM, March 2025, pp. 498–514.